2021 IEEE 3rd International Conference on Communications, Information System and Computer Engineering (CISCE 2021)

A Comprehensive Discussion on Deep Reinforcement Learning

Weikang Xu[†,*]
University of New South Wales
Sydney, New South Wales, Australia
*weikang.xu@unswalumni.com

Linbo Chen[†]
University of California at Berkeley
Berkeley, California
chenlinbo@berkeley.edu

Hongyu Yang[†]
University of Waterloo
Waterloo, Ontario, Canada
h79yang@uwaterloo.ca
[†]These authors contributed equally.

2021 International Conference on Communications, Information System and Computer Engineering (CISCE) | 978-1-6654-0352-8/21/$31.00 ©2021 IEEE | DOI: 10.1109/CISCE52179.2021.9445914

Abstract—Deep reinforcement learning (RL) is an important method to provide solutions for the perception and decision-making problems of complex systems. It has provided great contributions in natural language processing, drug design, gaming, and some other fields. This paper gives an overview of recent achievements in deep reinforcement learning. First, some basic concepts regarding deep learning, reinforcement learning and deep reinforcement learning are first introduced in this paper. Then, the milestone related work of deep RL and two methods of deep RL: deep Q-learning-based and policy gradient-based are reviewed. At last, an overview of deep RL applications is presented, and the future of deep RL is also proposed.

Keywords-deep learning; reinforcement learning; deep reinforcement learning; deep Q-learning; policy gradient

I. INTRODUCTION

Deep reinforcement learning is a branch of machine learning that combines deep learning and reinforcement learning [1]. It has the advantages of both deep learning and reinforcement learning to provide solutions for the perception and decision-making problems of complex systems. A simple classification method of reinforcement learning is shown in Fig. 1. Reinforcement learning can be divided into value-based (like Q-learning) method and policy-based method (like policy gradient). Similarly, deep RL can also be divided in this way [2].

Valued-based deep RL means that how good a given action or a given state is for the agent. In terms of rewards, selecting one action over another in a given state may increase (or decrease) the agents' return. Knowing this in advance can help the agent to decide which actions to take in which states in order to maximize the expected return.

Policy-based deep RL means how likely it is for an agent to take any given action from any given state, which means the probability that an agent selects a specific action from a specific state. Deep reinforcement learning has achieved remarkable results not only in academic fields: artificial intelligence, video games, healthcare, computer vision, but also in real application of AlphaGo, which is a computer program developed by Google DeepMind based on deep reinforcement learning and beat the world's top Go player Lee Sedol 4:1 out of five games in March 2016. Deep

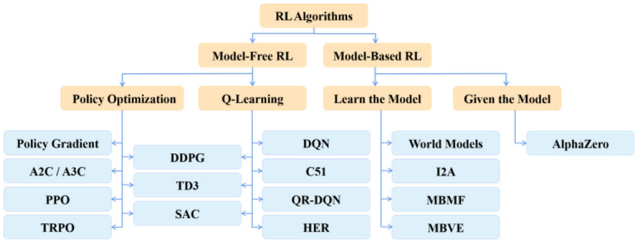

Figure 1. Reinforcement learning classification

reinforcement learning can be applied in autonomous driving like AWS DeepRacer as it can control various aspects like speed limit, collision, and direction. It can also be used for robotics manipulation, recommendation, natural language processing, and other fields [3].

This paper aims to give an overview of recent research about deep reinforcement learning with focuses on deep Q learning-based and policy gradient-based deep reinforcement learning. The remaining is organized as follows: Section 2 is a brief review of the concepts of deep learning, reinforcement learning, and deep reinforcement learning. Section 3 and Section 4 present deep Q learning and policy gradient, respectively. Section 5 presents the basic method of deep reinforcement learning and some improvements. Finally, Section 6 concludes the paper.

II. PRELIMINARIES

Generally, in deep learning, agents have strong perception ability but weak decision-making ability. By contrast, agents can select an optimal action but have a weak representation of the environment in reinforcement learning. Thus, finding a combination of deep learning and reinforcement learning is important. Deep reinforcement learning, which combines deep learning and reinforcement learning and complements the advantages of the two, is proposed. Both deep learning and reinforcement learning are subfields of machine learning, but they have different characters. Therefore, it is important that fully understanding deep learning and reinforcement learning when trying to understand what deep reinforcement learning can do, how it works, and how we use it.

A. Deep Learning

Deep learning allows models have multiple layers to learn a representation of input data with multiple levels of abstractions [4]. Those methods have solved many complex representation

978-0-7381-1215-2/21/$31.00 ©2021 IEEE

problems and have been used in data mining, machine learning, artificial intelligence, machine translation, game programs, and other fields. Deep neural networks allow the deep learning methods to be very powerful tools in solving the real problem [5-6]. The basic structure of the deep neural network is shown in Fig. 2.

The most famous deep neural networks include Convolutional Neural Networks (CNNs), Long Short-Term Memory Networks (LSTMs), Recurrent Neural Networks (RNNs), and Autoencoders. In the following sections, we will briefly introduce Convolutional Neural Networks (CNNs) [7] and Autoencoders [8].

Convolutional Neural Networks

CNNs contain multiple layers and mostly is used to analyze visual images. Fig. 3 shows a classical CNNs architecture for image processing. As shown in the figure, CNN has multiple layers which can process and extract features from input images.

The classical objective function is defined as

$$min\frac{1}{2}\sum_{j=1}^{nL}(r_j - a_j(L))^2\,, \tag{1}$$

where r_j represents labels and the $a_j(L)$ denotes the output of j-th neuron of L-th layer. The convolution layer has several filters to obtain features, then those features are passed to ReLU layer, and the output is a rectified feature map. It is worth mentioning that CNN uses backpropagation to train models, so a vanishing gradient may occur. However, ReLU layer can prevent vanishing gradient and the gradient does not decrease badly. After that, the rectified feature maps enter the pooling layer, and the dimensions are reduced naturally. The above three kinds of structures extract useful features but do not know what features belong to. Thus, the fully connected layer and loss layer are used to classify the features.

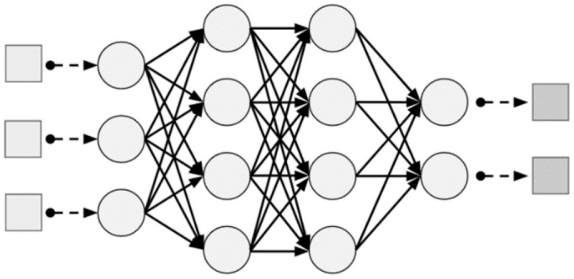

Figure 2. The basic structure of a deep neural network

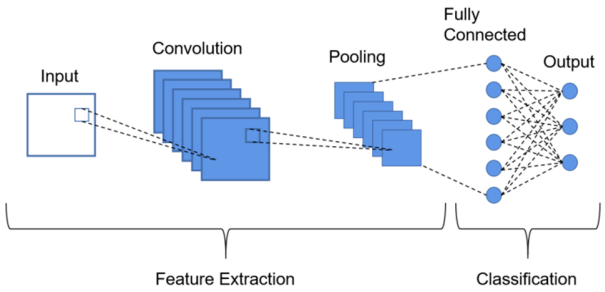

Figure 3. A typical CNN architecture for image processing

1) Autoencoders

The Autoencoder is a neural network with three or more layers, which encodes the input x into a new representation y, and then it decodes y back to x. This is an unsupervised learning algorithm, which uses back-propagation to train the network so that the output is equal to the input. The basic structure of Autoencoder is shown in Fig. 4.

The objective function of Autoencoder is defined as

$$\min\sum_{n=1}^{N}\left\|x^{(n)} - g(f(x^{(n)}))\right\|^2 \tag{2}$$

where $f(x)$ denotes the encoder function and $g(x)$ denotes the decoder function.

B. Reinforcement Learning

Reinforcement learning is one subfield of machine learning. It is used to decide how an agent should take action to gain maximum reward when the agent interacts with the environment. In Reinforcement learning, agents are trained based on an environment in which each move has a reward, and the agent will try to maximize the notion of cumulative reward. Reinforcement learning has been applied to multiple real-world applications, like industrial automation, finance and trading (like IBM), gaming, and even marketing and advertisement [9].

C. Deep Reinforcement Learning

Based on the above parts, the deep learning method has a strong representation of environment but weak decision-making ability, while reinforcement learning can make optimal decisions but has no perception ability. Therefore, it is important to find a way to represent the environment and choose the best action at the same time. Hence, deep reinforcement learning is presented [10].

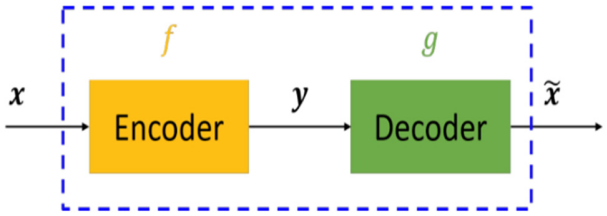

Figure 4. The basic structure of an autoencoder: The basic structure of an autoencoder: x is the input signal that requires to encode, f and g is the function of encoder and decoder.

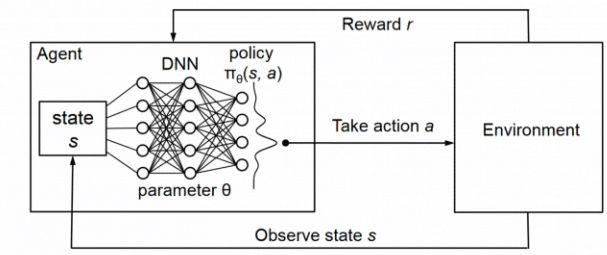

Figure 5. The framework of deep reinforcement learning

Deep reinforcement learning combines the perception ability of deep learning and the decision-making ability of reinforcement learning. Figure 5 shows a basic architecture of deep reinforcement learning: At each time step t, the agent interacts with the environment and observes state st where st ∈ S based on the deep learning method. Evaluate the reward function of each action and selects an action (a) from a set of actions $A := (1, 2, \ldots, k)$ when a is at state $\boldsymbol{s_t} \in \boldsymbol{S}$, where S is the set of states. As a result of the action, the agent will receive a reward $\boldsymbol{r_t} \in \boldsymbol{R}$. The environment reacts to this action, and the agent observes the next state $\boldsymbol{s_t + 1} \in \boldsymbol{S}$.

This is the normal way how deep reinforcement learning works. We will also introduce deep Q learning and policy

$$Q_n(x,a) = \begin{cases} (1-\alpha_n)Q_{n-1}(x,a) + \alpha_n[r_n + \gamma V_{n-1} \cdot y_n] \;\; if \;\; x = x_n \;\; and \;\; a = a_n, \\ Q_{n-1}(x,a) \qquad\qquad otherwise, \end{cases} \tag{3}$$

where the value function is:

$$V_{n-1}(y) \equiv \max_b \{Q_{n-1}(y,b)\}. \tag{4}$$

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Figure 6. Deep Q-learning with Experience Reply

The above Q-learning algorithm had been around for many years as one of the most important base concepts in the realm of RL.

However, it was shown to have major room for improvements as more and more researchers discovered defects in Q-learning. Bellemare et al. proposed a family of optimality-preserving operators on Q-functions which were able to increase the action gap in each state and reduce error in greedy policies. Later, researchers made a simple change to the Bellman operators and introduced a notable operator among the group of operators. By doing so, the researchers were able to increase performance significantly, which shows the unreliable nature of traditional classical Q-function when practiced [11]. With the advancements in deep learning over the years and the idea of deep reinforcement learning emerging, researchers from Google DeepMind pioneered the first deep learning model in [20] to perform reinforcement learning "directly from high-dimensional sensory input". The architecture, called DQN, builds upon the Q-learning algorithm to include deep learning in order to estimate the action-value Q-function. We can use such a function approximator, called Q-network that is based gradient along with their improvements in the following sections.

III. Related Work of Deep Q-learning Network

In this Section, we review representative works of DQN. The traditional and foundational reinforcement learning concept of Q-learning, was proposed by Chris Watkins in 1989 and then presented in [11]. Q-learning is proved to optimize action-values in controlled Markovian domains. The main idea of Q-learning involves calculating a Q-value as well as a Value function at each state n, and adjusting such calculation for enough stages to achieve convergence:

upon some Neural Network architecture with weights θ and a loss function $L_i(\theta_i)$,

$$L_i(\theta_i) = \mathbb{E}_{s,a\sim p(\cdot)}(y_i - Q(s,a;\theta_i)^2). \tag{5}$$

To achieve convergence of Q-value expediently by calculating the gradient $\nabla_{\theta_i} L_i(\theta_i)$ and performing stochastic gradient descent on it. The model borrows concepts from a similar prior work called neural fitted Q-learning (NFQ), combining a Convolutional Neural Network (CNN) and a variant of Q-learning to play at a state-of-the-art level in seven Atari-2600 games. As discussed above and shown in the Fig. 6, gradient descent as well as several useful techniques including the Experience Replay (ER) are used in the first DQN model. We will discuss the details of the study and improvements on some of those techniques, which resulted in improved variants of this vanilla DQN model, in the following paragraphs.

The first technique in DQN to talk about is gradient descent. A study in the work [2] was one of the first to find promising qualities of gradient descent in Reinforcement Learning. In the paper, Baird showed at least three superior properties of using gradient descent in the field of Reinforcement Learning: (1) In contrast to the standard algorithms, A family of algorithms based on stochastic gradient descent guarantee convergence to a local minimum at the time diverging on even very a simple and small Markov decision model; (2) Gradient descent has practical benefits in speed by converging inelegant algorithms like Advantage Updating into a simpler and faster Advantage Learning; (3) Gradient descent allows for the creation of entirely new classes of RL algorithms such as VAPS that can combine two different RL approaches (values and policy search) into one unified algorithm. The work [12] pointed out the significant theoretical and practical values of gradient descent in the field for future research.

Another technique used in DQN called Experience Replay was shown by Lin in [22] to break the temporal-correlation problem of online RL agents in their simplest form. By using ER to store past experiences and replaying them, RL agents are able to main the i.i.d. assumption of the experiences and also use rare experiences for more than just one update. In the work [13], Schaul et al. improved further upon the existing

experience replay by introducing prioritized experience replay. Instead of replaying past transitions randomly, PER selects only the ones with high expected learning progress, as measured by their temporal-difference (TD) error, more frequently. This improvement made DQN more efficient and DQN with PER was able to achieve a new state-of-the-art by outperforming the vanilla DQN on 41 out of 49 games.

The invention of DQN sparked many attempts to improve upon it by making changes to the algorithm itself. This paper [14] creates a new variant of DQN called the Dynamic Frameskip Deep Q-Network that allows for the choice of frameskip value k, which determines how many times an action would be repeated on the current state. Whereas normal DQN or Dueling DQN just use a fixed number of repetitions. The paper shows by experiments that setting this variable is able to improve the performance of DQN on relatively harder games [14]. With an attempt to understand and improve DQN, this paper [16] shows that by introducing a discount factor gamma and adapting the learning rate, the convergence of DQN is able to speed up, thus improving its performance. Google DeepMind researchers also adopted adaptive normalization in their efforts to improve DQN to play Atari games. With adaptive normalization, they removed the clipped reward, which is predetermined to a certain range, from their prior DQN model and prevented the loss in performance due to this domain-specific heuristic [15].

Two variants of DQN:double DQN and Dueling DQN, are also very worth mentioning. Double Q-learning algorithm, developed by van Hasselt et al. from Google DeepMind, is a major improvement upon the original DQN algorithm and serves as a foundational work for future DQN variants such as the Dueling DQN. In paper [19], the authors show that the DQN algorithm by itself suffers from substantial overestimations. The Double DQN, using two same but separate Deep Neural Networks for the different tasks of policy evaluation and policy search, is able to reduce overestimation and improve performance on Atari-2600 games.

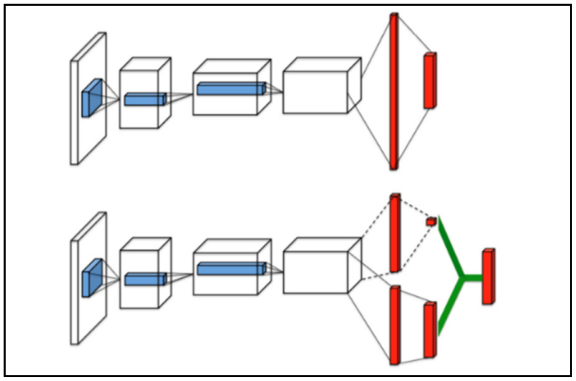

Figure 7. A popular single-stream Q-network (top) and the dueling Q-network (bottom).

The Dueling DQN, also developed by Google DeepMind researchers, serves as a further improvement on Double DQN by introducing the theoretical concept that Q function is equal to the summation of the state value function and a state-dependent action advantage function: Instead of using just a single stream at the end of the architecture to represent the Q-function layer, the Double DQN uses two separate streams, one for the state value and one for the advantage function and then combining them to represent the Q-value function. This new architecture was able to generalize learning across actions without imposing any change to the given architecture, and it was able to achieve a new state-of-the-art on the Atari 2600 games.

IV. A SURVEY OF DEEP REINFORCEMENT LEARNING BASED ON STRATEGY GRADIENT

The value-based approach has been approved effectiveness in certain fields, however, there are still limitations to value-based methods. First, it can hardly handle stochastic situations because the value-based approach is designed toward finding deterministic policies. Second, the value-based method is less efficiency when dealing with continuous action-spaces because it needs to calculate the maximum value and decide which action to take, but computational cost is so expensive in continuous action space. Therefore, policy-based methods are introduced to solve those kinds of problems. Rather than developing a value function, the policy-based method tries to improve policy itself directly by using an independent function approximator with its own parameters.

$$\nabla\theta \approx \alpha \frac{\partial \rho}{\partial \theta}, \tag{6}$$

where θ and ρ represents the vector of policy parameters and performance respectively, and α is a positive-definite step size. By using this method, a locally optimal policy can be guaranteed reached. To solve this equation, AT&T Labs proposed a policy gradient theorem in the work [23].

Theorem (Policy Gradient). for any MDP, in either the average-reward or start-state formulations as Equation (7),

$$\frac{\partial \rho}{\partial \theta} = \sum_s d^{\pi}(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta} Q^{\pi}(s,a), \tag{7}$$

$d^{\pi}(s)$ is the stationary distribution of states under π, $d^{\pi}(s) = \lim_{t\to\infty} \Pr\{s_t = s \mid s_0, \pi\}$.

$Q^{\pi}(s,a)$ is the value of a state-action pair. Then they proved that learned function approximator $f_w(s,a)$ can replace $Q^{\pi}(s,a)$. And since advantage function can be consider as $A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$. We can finally simplify the equation as follows:

$$\frac{\partial \rho}{\partial \Theta} = \sum_s d^{\pi}(s) \sum_a \frac{\partial \pi(s,a)}{\partial \theta} A^{\pi}(s,a) \tag{8}$$

After few step transformations, the most common gradient estimator is derived as follows:

$$\hat{g} = \hat{\mathbb{E}}_t\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t)\hat{A}_t\right], \tag{9}$$

where π_θ is a stochastic policy and $\hat{A}_t$ is an estimator of the advantage function at timestep t. This method reduces the computation of the performance gradient to simple expectation.

Also, if we use an action-value function $Q^w(s,a)$ with parameter vector w instead of $Q^\pi(s,a)$, and a critic using an appropriate policy evaluation algorithm (e.g., temporal-difference learning) to estimates this action function, we will get a new form as Equation (9):

$$\hat{g} = \hat{\mathbb{E}}_t[\nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^w(s,a)] \tag{10}$$

This algorithm is called stochastic "actor-critic". However, replacing $Q^\pi(s,a)$ may introduce bias, so the function approximator needs to be compatible to avoid bias. There are more methods developed base on this policy gradient methods, such as Deterministic Policy Gradient (DPG), Deep Deterministic Policy Gradient (DDPG), "vanilla" policy gradient methods, and trust region/natural policy gradient methods.

Deterministic Policy Gradient (DPG) was developed base on stochastic policy gradient theorem and stochastic actor-critic algorithms. The basic idea about DPG is that using stochastic behavior to choose action, but to learn a deterministic target policy to improve efficiency. DPG introduces an off-policy learning algorithm to ensure that DPG can explore the full state and action space same as stochastic policy gradient. Then DPG derives an off-policy actor-critic algorithm and updates the policy parameters in the direction of the approximate action-value gradient. Compares with stochastic policy gradient, Deterministic Policy Gradient (DPG) has higher efficiency because stochastic policy gradient usually requires large samples when action space is high-dimensional.

On the basis of DPG and the inspiration of DQN, Deep Deterministic Policy Gradient (DDPG) was introduced to handle continuous action spaces. DDPG is an actor-critic, model-free algorithm. DDPG used a neural network function approximator to learn large spaces and actions online [24].

Randomly initialize critic network $Q(s,a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N}\sum_i \nabla_a Q(s,a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu}\mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

Figure 8. The pseudo-code of DDPG algorithm

There are few problems related to this approach. First, most of the neural network assumes that samples are independently and distributed identically, which is not true in real-time environments. David Silver et al. used a replay buffer to solve this problem. Secondly, using a neural network to implement Q-learning directly has been proved unstable. Therefore, DDPG can slowly chooses to improve the target value to increase its stability. In practice, the importance of stability has outweighed the speed of learning. And also, exploration has always been a major problem for continuous action and space algorithms. David Silver et al. solved this problem by adding Gaussian noise sampled from a noise process to the existing policy.

Trust Region Policy Optimization (TRPO) is an optimization algorithm which optimizes large non-linear policy such as neural network [25]. This algorithm is based on proof that policy can be improved by minimizing a certain surrogate loss function. This algorithm has two variants, the single-path method, which is for the model-free setting. and vine method, which is only possible in simulation. This algorithm reaches the same performance in Atari games as other methods, but this algorithm has higher generality since we can apply the same policy search method to different domains. Therefore, this algorithm reaches remarkable achievements in the domain of robotic locomotion, such as swimming, walking, and hopping in a physics simulator. However, TRPO is hard to understand and implement. Therefore, John Schulman et al developed a new algorithm base on TRPO, called proximal policy optimization (PPO). The advantages of PPO are, easy to implement, more general, and have better sample complexity. PPO has better performance than other online policy gradient methods.

V. Conclusion and Future Works

Deep reinforcement learning (deep RL) has provided great contributions in natural language processing, drug design, gaming, and some other fields. In this paper, the detailed algorithms regarding the milestone relative work of deep RL are presented. Moreover, the paper discussed and reviewed two main methods of deep RL, which are the value-based method deep Q-learning and the policy-based method policy gradient.

The following thing we plan to do is to apply deep reinforcement learning (deep RL) in computer vision and natural language processing fields. Since the development of the two areas can improve robotic which has been used for specific tasks and make more benefits [26]. Deep RL has outperform passive vision systems although deep learning algorithms are well developed [27]. Natural language processing is another important application of deep RL. We expect that deep RL systems which could understand documents will be better when systems learn strategies focusing on only one part [28].

References

[1] François-Lavet V, Henderson P, Islam R, et al. An introduction to deep reinforcement learning[J]. arXiv preprint arXiv:1811.12560, 2018.

[2] Henderson P, Islam R, Bachman P, et al. Deep reinforcement learning that matters[C]//Proceedings of the AAAI Conference on Artificial Intelligence. 2018, 32(1).

[3] Arulkumaran K, Deisenroth M P, Brundage M, et al. Deep reinforcement learning: A brief survey[J]. IEEE Signal Processing Magazine, 2017, 34(6): 26-38.

[4] LeCun Y, Bengio Y, Hinton G. Deep learning[J]. nature, 2015, 521(7553): 436-444.

[5] Deng L, Yu D. Deep learning: methods and applications[J]. Foundations and trends in signal processing, 2014, 7(3–4): 197-387.

[6] Ngiam J, Khosla A, Kim M, et al. Multimodal deep learning[C]//ICML. 2011.

[7] Zhuang B, Shen C, Tan M, et al. Towards effective low-bitwidth convolutional neural networks[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2018: 7920-7928.

[8] Liou C Y, Cheng W C, Liou J W, et al. Autoencoder for words[J]. Neurocomputing, 2014, 139: 84-96.

[9] Wiering M, Van Otterlo M. Reinforcement learning[J]. Adaptation, learning, and optimization, 2012, 12(3).

[10] Kaelbling L P, Littman M L, Moore A W. Reinforcement learning: A survey[J]. Journal of artificial intelligence research, 1996, 4: 237-285.

[11] Bellemare M G, Ostrovski G, Guez A, et al. Increasing the action gap: new operators for reinforcement learning//Proceedings of the AAAI Conference on Artificial Intelligence. Phoenix, USA, 2016: 1476-1483

[12] Baird III L C. Reinforcement learning through gradient descent. Carnegie Mellon University, USA, 1999

[13] Schaul T, Quan J, Antonoglou I, Silver D. Prioritized experience replay//Proceedings of the 4th International Conference on Learning Representations. San Juan, Puerto Rico, 2016:322-355

[14] Lakshminarayanan A S, Sharma S, Ravindran B. Dynamic frame skip deep q network//Proceedings of the Workshops at the International oint Conference on Artificial Intelligence. New York, USA, 2016

[15] Hasselt H V, Guez A, Hessel M, et al. Learning functions across many orders of magnitudes//Proceedings of the Advances in Neural Information Processing Systems. Barcelona, Spain, 2016:80-99

[16] François-Lavet V, Fonteneau R, Ernst D. How to discount deep reinforcement learning: towards new dynamic strategies// Proceedings of the Workshops at the Advances in Neural Information Processing Systems. Montreal, Canada, 2015:107-116

[17] Wang Z, Freitas N D, Lanctot M. Dueling network architectures for deep reinforcement learning//Proceedings of the International Conference on Machine Learning. New York, USA, 2016: 1995-2003

[18] Hausknecht M, Stone P. Deep recurrent q-learning for partially observable MDPs. arXiv preprint arXiv:1507.06527, 2015

[19] van Hasselt H, Guez A, Silver D. Deep Reinforcement Learning with Double Q-learning. arXiv preprint arXiv:1509.06461. 2015 Sep 22.

[20] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., Riedmiller, M. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602, 2013.

[21] Watkins, C.J.C.H., Dayan, P. Q-learning. Mach Learn 8, 279–292 (1992). https://doi.org/10.1007/BF00992698

[22] Lin, Long-Ji. Self-improving reactive agents based on reinforcement learning, planning and teaching. Machine learning, 8(3-4):293–321, 1992.

[23] Duan J, Shi D, Diao R, et al. Deep-reinforcement-learning-based autonomous voltage control for power grid operations[J]. IEEE Transactions on Power Systems, 2019, 35(1): 814-817.

[24] Silver D, Lever G, Heess N, et al. Deterministic policy gradient algorithms//Proceedings of the International Conference on Machine Learning. Beijing, China, 2014: 387-395

[25] Schulman J, Levine S, Moritz P, et al. Trust region policy optimization//Proceedings of the International Conference on Machine Learning. Lugano, Switzerland, 2015: 1889-1897.

[26] DRAPER, R. SMART ROBOTS, A HANDBOOK OF INTELLIGENT ROBOTIC SYSTEMS-HUNT, VD. 1985: 46-52.

[27] Ba, Jimmy, Volodymyr Mnih, and Koray Kavukcuoglu. Multiple object recognition with visual attention. arXiv preprint arXiv:1412.7755, 2014.

[28] LeCun, Yann, Yoshua Bengio, and Geoffrey Hinton. Deep learning. nature 521.7553 2015: 436-444.